CAFE ORDERING SYSTEM

Object-Oriented Programming – Cafe Ordering System

Section: ITS-201 – Section C

Date: November 9, 2026

Submitted by

Romeo E. Pamplona

Adrian R. Dais

OOP Concepts Used in the Program

1. if Statement

We used if statements in Pastry.computePrice() to add an extra charge when a pastry is warmed and in Meal.computePrice() when a meal is upsized. We also used if statements in Order to check whether an item slot contains an item before calculating its price.

2. if-else Statement

We used if-else statements when the program needs to choose between two outcomes. Examples include validating the base price in Menu_item.setBasePrice(), checking beverage sizes in Beverage.computePrice(), limiting the number of items in Order.addItem(), and preventing a negative total in Order.computeTotal(double).

3. Classes and Objects

Our program has seven classes: CafeApp, Order, Customer, Menu_item, Beverage, Pastry, and Meal. We create customer, menu item, and order objects in CafeApp.main() to run the cafe ordering process.

4. Attributes and Behaviors

Each class has attributes that store information and methods that describe its actions. For example, Beverage stores its size, Pastry stores whether it is warmed, and Order has methods such as addItem(), computeTotal(), and printReceipt().

5. Object Interaction

The Order class stores a Customer object and up to three Menu_item objects. When printReceipt() is called, Order gets customer and item information, calculates prices and discounts, prints the receipt, and updates the customer's loyalty points.

6. Java Naming Conventions

We use readable class, method, and variable names. Methods and variables follow camelCase, such as computeTotal() and basePrice. The current parent class is named Menu_item.

7. Default Constructor

We use default constructors in Menu_item, Beverage, Pastry, Meal, and Customer to give objects starting values when no information is provided. For example, the default Beverage constructor sets the size to S.

8. Parameterized Constructor

We use parameterized constructors to provide object information when an object is created. The Beverage, Pastry, and Meal constructors use super() to pass the common name, base price, and item code to the parent class.

9. Constructor Overloading

The Customer class has three constructors: one without arguments, one that accepts a name, and one that accepts a name and customer type. This lets us create customers in different ways.

10. public, private, and protected

We use private fields to protect data, public methods to allow other classes to use class behaviors, and protected name and basePrice fields in Menu_item so its subclasses can access them.

11. Getters, Setters, and Data Hiding

We use getters and setters to read and update attributes instead of making fields public. Validation in setters prevents invalid values such as negative base prices, negative extra sides, and negative loyalty points.

12. Method Overloading

We use addPoints(int) and addPoints(double) in Customer to add points in two ways. In Order, computeTotal() calculates the discounted total, while computeTotal(double voucher) also applies a voucher.

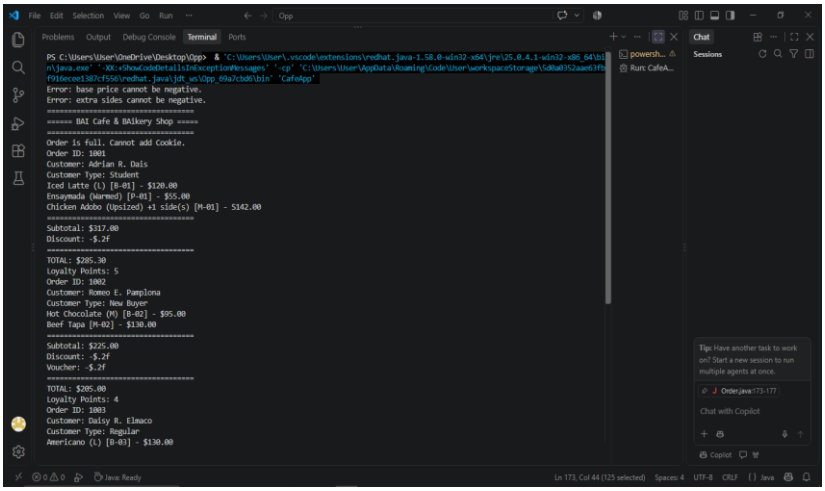
13. Method Overriding

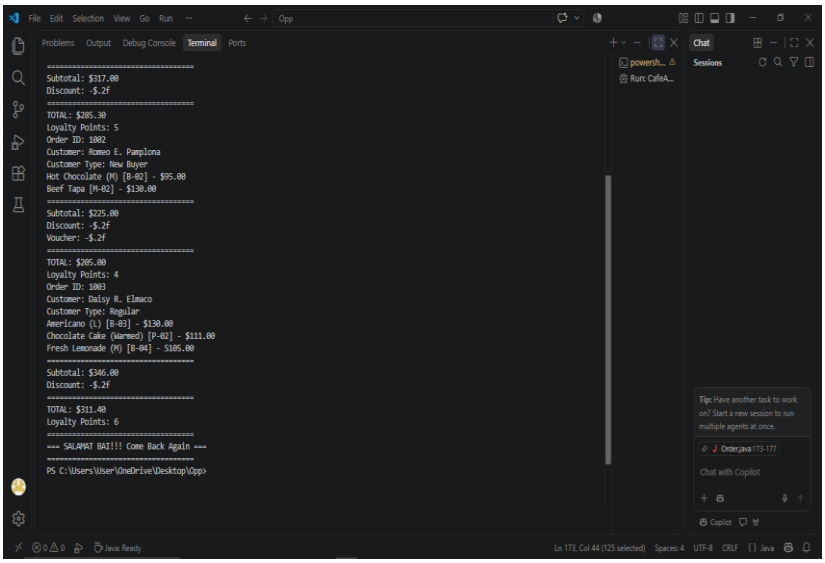
Beverage, Pastry, and Meal each provide their own versions of computePrice() and getDescription(). We use the @Override annotation to show that these methods replace inherited behavior from Menu_item.

14. Abstraction

Menu_item is an abstract parent class with an abstract computePrice() method. Beverage, Pastry, and Meal implement that method using their own pricing rules.

ScreenShot/Output





Challenges Encountered and Solutions

During the development of the program, we encountered some challenges, especially in using constructors, inheritance, and method overloading.

One challenge was creating different customer types, such as students, regular customers, and new buyers. We solved this by using constructors and customer type methods to identify each customer.

Another challenge was calculating the correct prices and discounts. We used conditional statements to apply additional charges for large beverages, warmed pastries, upsized meals, and customer discounts.

We also encountered problems with negative values and loyalty points. We solved these by adding validation inside setters and methods to prevent invalid information from being accepted.Lastly, we improved the receipt output to display the customer's information, ordered items, subtotal, discount, voucher, total amount, and loyalty points.